rogii-wellbore-geology-prediction

2nd Place Solution: AnchorCNN — Conditional Probabilistic Path Modeling

Rank	2
Team	Bilzard
Author	Bilzard
Topic ID	733432
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733432
Retrieved with Kaggle CLI on 2026-08-10.

First of all, thank you to the hosts for organizing this competition. (I also thank my faithful partner Claude Code Agent :D)

My public LB rank kept stagnating, which made the final stretch a really tough fight. I wanted as many people as possible to know about this solution, so I was determined to finish near the top. I kept training models until 3 hours before the deadline, and I'm happy that it resulted in 2nd place on the private LB.

TL;DR

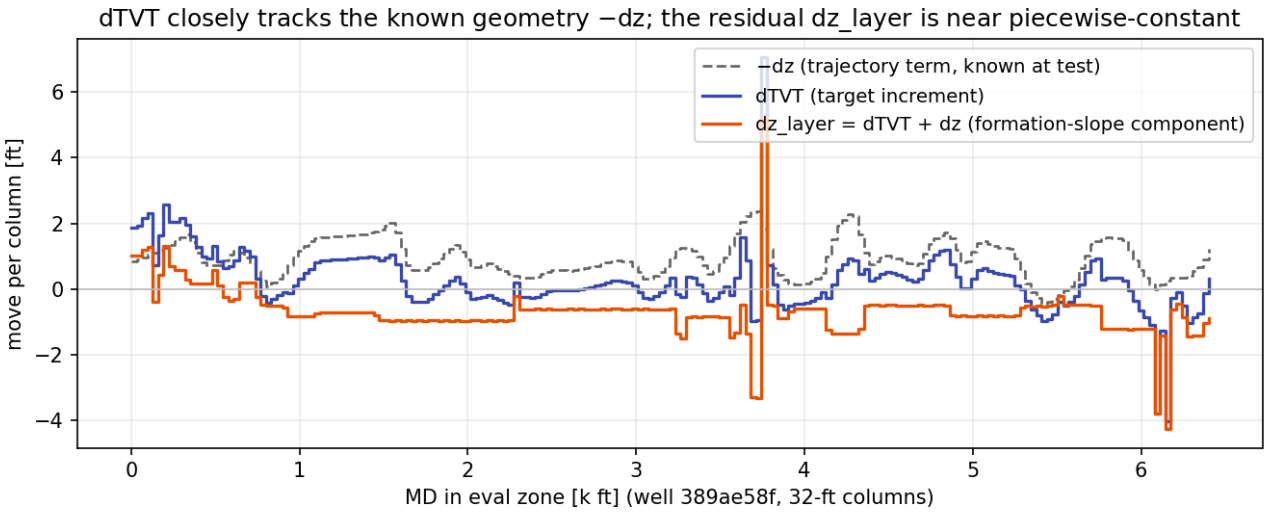
I trained an anchor-grid CNN that predicts conditional move distributions P(dTVT|TVT) on physically consistent synthetic data, decoded the expected path exactly with dynamic programming, and selected models with validation designed to be robust to a handful of dominant wells.

Key insight from the data

The formation surfaces interpreted by experts (the `ANCC`, `BUDA`, etc. columns in the train data) have a characteristic "sequence of straight lines" shape. Writing z_layer for the depth of the formation surface (so that its increment dz_layer is the formation-slope component), the identity

```
dTVT = dz_layer − dz    (1)
```

holds (it follows directly if you define z_layer := TVT + z − b_well; z is the measured TVD of the trajectory and is known for every row even at test time). Looking at real samples, dz_layer/dMD is staircase-shaped: constant over most intervals, with occasional steps. Since the identity fixes the coefficient at −1, **wherever there is no step, the shape of dTVT matches −dz exactly** (this is why the two curves run parallel in the figure below; steps appear in only about 10% of the columns).



→ By feeding dz/dMD (dip z) as an input feature, the model's job shrinks to "estimating the smooth structural slope dz_layer", strongly steering it toward formation surfaces that experts would plausibly draw.

Synthetic data

My synthetic data generation rests on the following **generative assumption** (a layer-cake idealization: rocks at the same stratigraphic depth share the same properties):

```
GR_obs = f(TVT) + r(TVT)    (2)
```

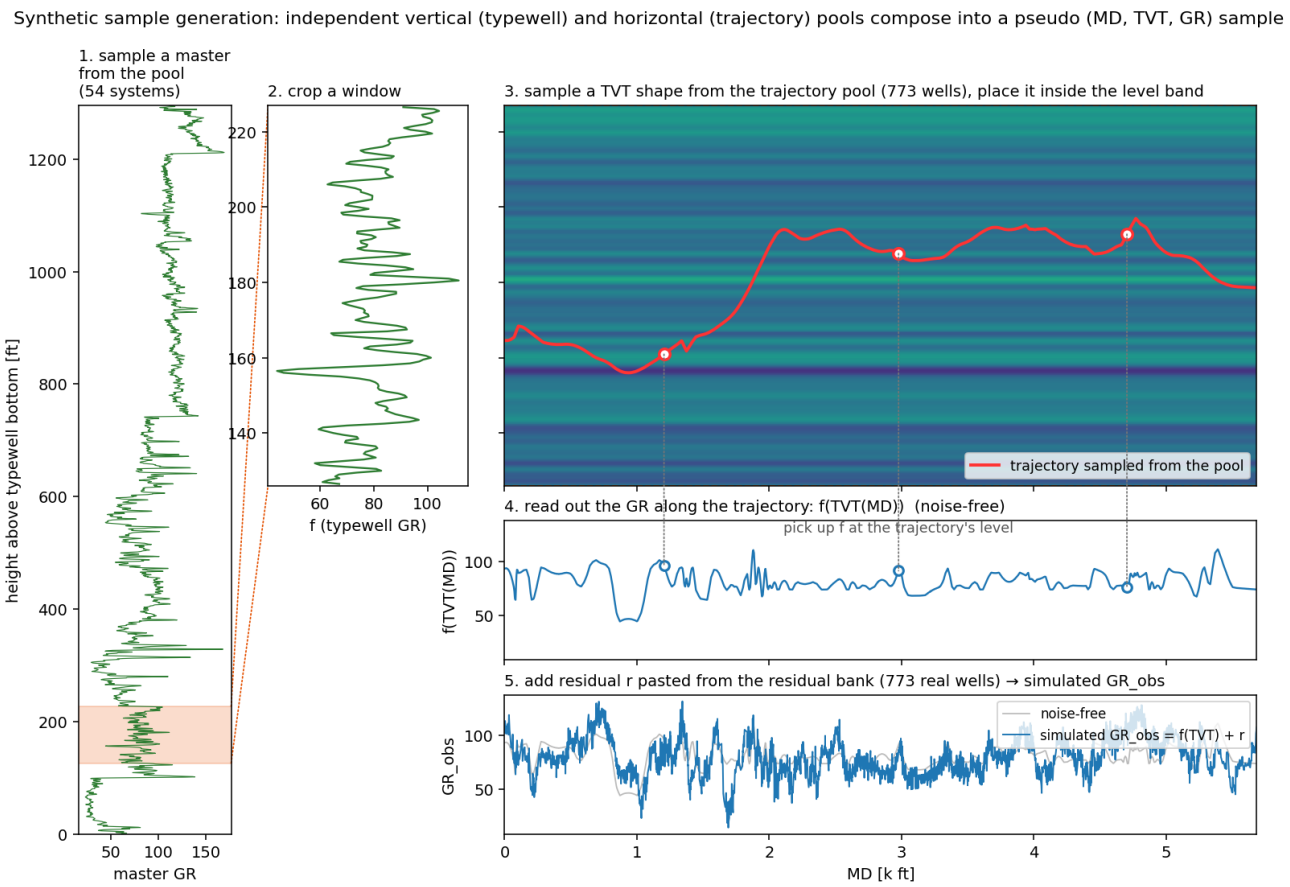
- f: the TVT–GR profile of the typewell
- r: residual noise

Since (2) is a function of TVT, it has a remarkable property: **given any TVT trajectory, you can generate an unlimited amount of data consistent with this assumption**. Where the assumption breaks (GR deviating per well even at the same TVT), I measured the deviation from real data and carried it into the synthesis through the residual r, as described below.

Data synthesis pipeline

The vertical material (the typewell's level→GR profile) and the horizontal material (the TVT trajectory along MD) are sampled from independent pools, and composed by reading out f(TVT) to form a pseudo (MD, TVT, GR) sample:

1. Sample one master from the master pool (54 systems)
2. Crop a window = the typewell profile f
3. Sample a TVT trajectory shape from the real-well pool (horizontal pool) and place it at a depth inside the level band
4. Read out the GR along the trajectory: f(TVT(MD)) (noise-free)
5. Add a residual r sampled from the residual bank (773 real wells) = the simulated (noisy) GR_obs



The next three subsections detail these stages in pipeline order.

Synthesizing typewell GR (steps 1–2)

As pointed out in a discussion comment [3], the train typewells can be consolidated into a small number of master well series. My own verification consolidated them into **54 systems**, confirming that every typewell in a system is fully contained in a single master TVT–GR series. Cropping arbitrary windows from these master series generates a typewell profile f for any level range.

Generating TVT trajectories (step 3)

Feeding curves that could never appear in train makes the model learn spurious features, so instead of free-form curves, trajectories are generated **in two steps from real train trajectories**.

(a) Shape: deform the shape of real trajectories with vertical shifts and mixup of two trajectories:

```
TVT_mix = λ·TVT1 + (1−λ)·TVT2    (3)
```

(b) Placement into a system (re-skinning): assign the shape to a typewell system and decide the depth to place it at. The quantile the trajectory occupied within the drilling band of its source system (the distribution of levels where wells of that system are actually drilled) is mapped to the same quantile of the target system's drilling band. The shape is preserved, the depth becomes realistic for the target system, and the GR is re-generated from the target system's master series (= a new "skin").

> Note (consistency of z): while transforming trajectories, **z_layer — not z — is kept as the internal representation**, and z is derived at the very end as z = z_layer − TVT. This keeps identity (1) consistent by construction, no matter what transform is applied.

Adding residuals (step 5)

Examining the residual r = GR_obs − f(TVT) of real wells, it decomposes into **a smooth well-specific mean profile that emerges when you aggregate over TVT**, plus a remainder:

```
r(TVT) = δ_w(TVT) + ϵ    (4)
```

- δ_w: the systematic component — the well-specific signature of "how much this well's GR deviates from the typewell at each depth" (the mean of r aggregated over TVT)
- ε: the remaining non-systematic noise; not white, it carries autocorrelation

For each synthetic well, noise was injected in one of two ways:

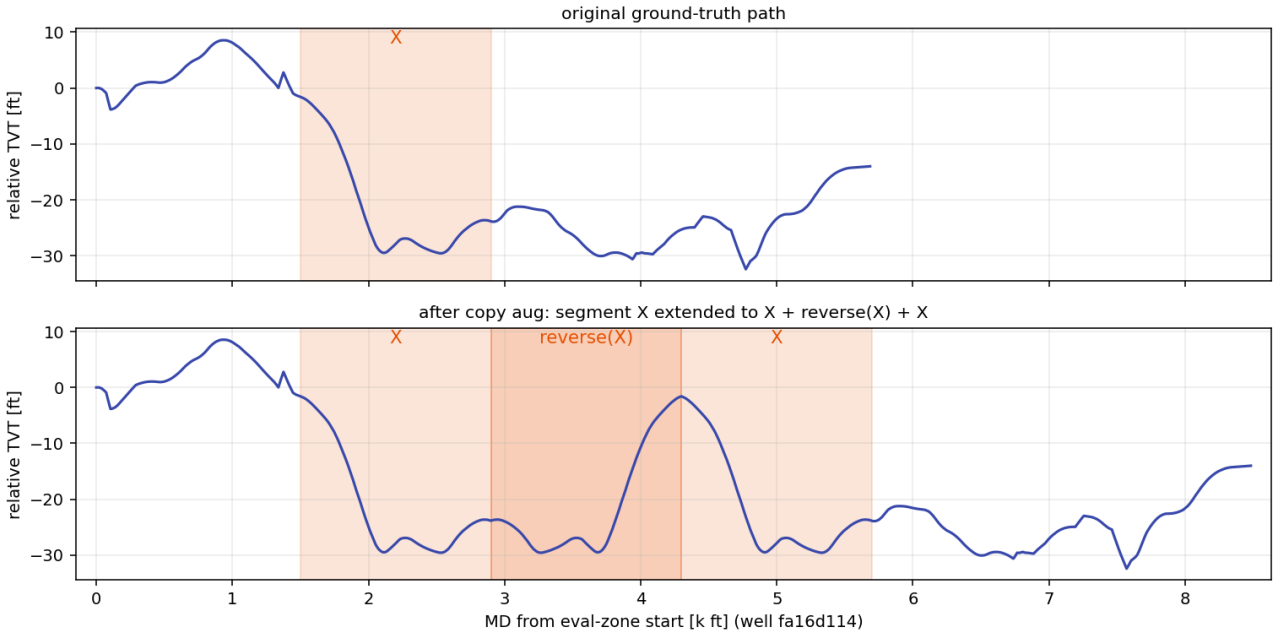
- **Pasting real residuals**: paste a real well's residual sequence r as-is (this carries over both δ_w and ε with all their real-data properties)
- **Synthesis**: draw δ_w as a smooth random function of TVT, and mimic ε with an AR noise model fitted to real data

This produced synthetic data that carries realistic "well-specific GR profile quirks".

Augmentation

Augmentations were applied to both real and synthetic wells at training time. The main ones:

- **Geometric**: MD flip, level flip (with label sign flip), random crop along MD, level shifts (±32 ft)
- **Structural**: flips and scaling in z_layer space (stretching the structural slope). Consistency with identity (1) is preserved by the z_layer internal representation
- **Length normalization and stitching**: short wells are extended to a target length by copy&paste of themselves or fragments of other wells, with seams smoothly joined by short splines (100 ft). I also used a copy aug that redundantly extends a segment inside the eval zone to X + reverse(X) + X (figure below; values stay continuous at the seams, only the gradient flips)
- **Observation quality**: specaug that masks spans of GR (up to 21 columns) in exactly the same format as real missing data



Modeling

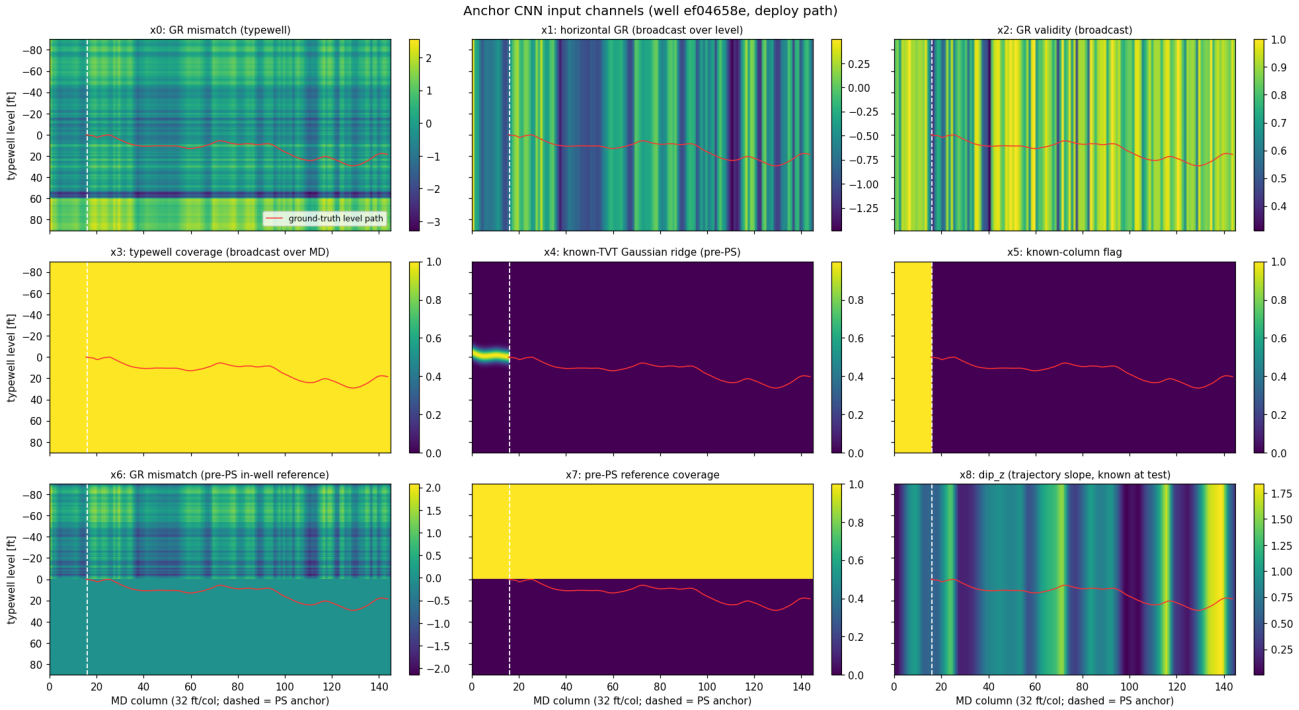
I started from the CNN that @hengck23 shared in the discussion [1] (unfortunately, the paper's authors had not released a detailed implementation of the model [2]).

GR matching in this task is fundamentally ambiguous. The typewell GR profile contains repeating, similar-looking stripes, so locally, **multiple level hypotheses — assumptions that "the well is at this depth right now" — match the GR equally well**. In other words, the posterior is multimodal, and there are multiple plausible trajectory candidates (multi-path). I confirmed this in early experiments: a model that regresses a single teacher path indeed collapses onto the average of these modes — a path that corresponds to none of the hypotheses — and is therefore mismatched with the task.

My reading of [2] is that its essence is to model multi-path as a conditional probability model P(dTVT|TVT): holding, **for every level hypothesis simultaneously**, the distribution of "if the well is at depth TVT now, how much does it move up or down over the next interval". So I designed a probability model that expresses this explicitly.

Input features

The input is a (9, 512, 336) image tensor: vertical = typewell level (0.5 ft/bin, ±128 ft), horizontal = MD (32 ft/column; the first 16 columns are the known pre-PS interval).



The 9 channels (x6–x8 were added late in the competition, so their gains are measured):

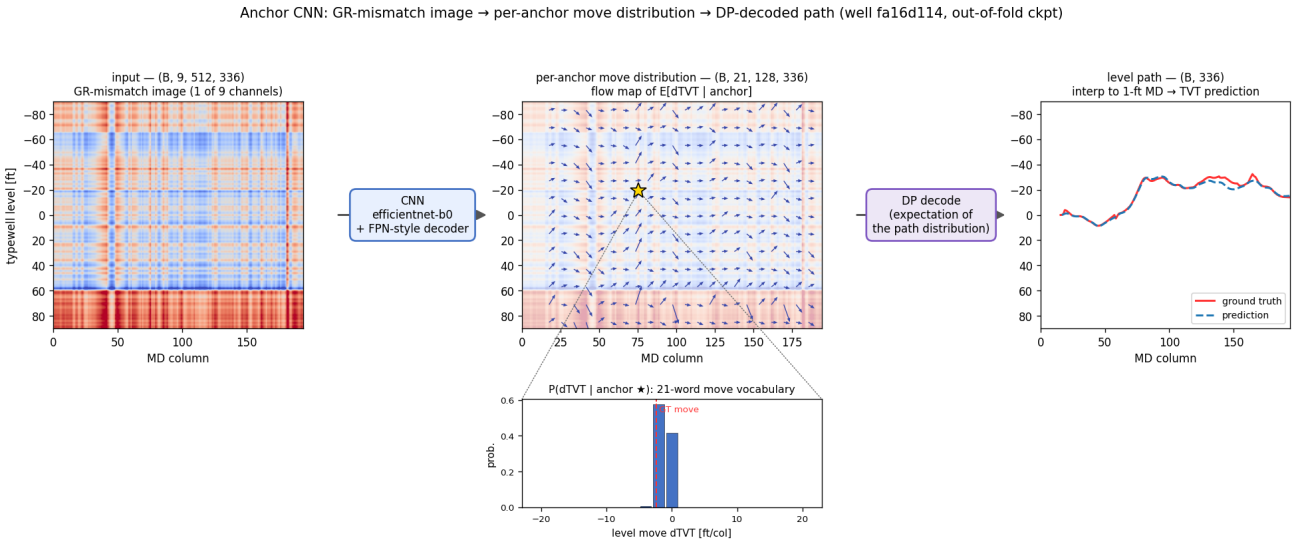
ch	content	gain
x0	GR mismatch (typewell): for each (level, MD), the discrepancy between the lateral GR and the typewell GR assuming the well is at that level	
x1	lateral GR (broadcast over level)	
x2	GR validity (broadcast)	
x3	typewell coverage (broadcast over MD)	
x4	Gaussian ridge of known TVT (TVT_input prior; the known pre-PS interval is drawn into the first 16 columns)	
x5	known-column flag	
x6	GR mismatch (pre-PS): a matching image against a level→GR reference built from the same well's pre-PS interval (inspired by the host's tip [6])	together with x7: OOF −0.759 / public −0.271 / private −0.320
x7	coverage mask for x6	(paired with x6)
x8	dip z: per-column slope of the trajectory z (known at test time)	public −1.089 / private −0.470 (*)

The x6/x7 gains are the clean difference of a submission pair (sub061 → 062) where only this channel changed. x8 (*) is a mixed effect of three changes made at once: adding the channel, revising the structural augmentation, and switching the decoder to DP-only.

AnchorCNN

The basic design philosophy takes inspiration from the anchor-based / anchor-free family of object detectors (YOLO [4], CenterNet [5], etc.). CNNs are good at encoding local features, and the move direction dTVT at a specific coordinate (MD, TVT) is easy to predict from local features. So I mapped the **conditioning part — "at coordinate (MD, TVT)" — onto CNN anchors**. Here an **anchor** means the embedding vector at coordinate (i, j) of the CNN feature map. Coordinate (i, j) corresponds to a level hypothesis (being at level i in MD column j), and from each anchor the model reads out a move distribution conditioned on that position.

Concretely, the outputs of every efficientnet-b0 stage are fused by an FPN-style decoder (each stage projected to a common width by 1×1 conv, interpolated to a single resolution, and summed), then mapped onto a **(B, 21, 128, 336) state grid** (vertical = level at 2 ft/bin, horizontal = 32 ft/column). From the anchor at each grid point, the model predicts the discrete move distribution P(dTVT | anchor). The move vocabulary — the values dTVT can take — has 21 classes: {0, ±2, ±4, …, ±20} ft/column (i.e. {−10, …, +10} bins of 2 ft).



Training

Supervision is teacher-forced along the ground-truth path: at each column, the anchor lying on the ground-truth level receives a cross-entropy loss against the quantized ground-truth move (plus an auxiliary intra-bin position loss). Each epoch mixes the real training wells (~620 per fold) with 2,048 synthetic wells generated on the fly. Models were trained for 120 epochs with a constant learning rate of 1e-3 after warmup (WSD schedule with no decay) and an EMA of the weights. The full 5 folds × 3 seeds took about 10 hours on a single GPU.

Path generation

Starting from the PS point, there are two ways to generate paths from the probability field on the state grid:

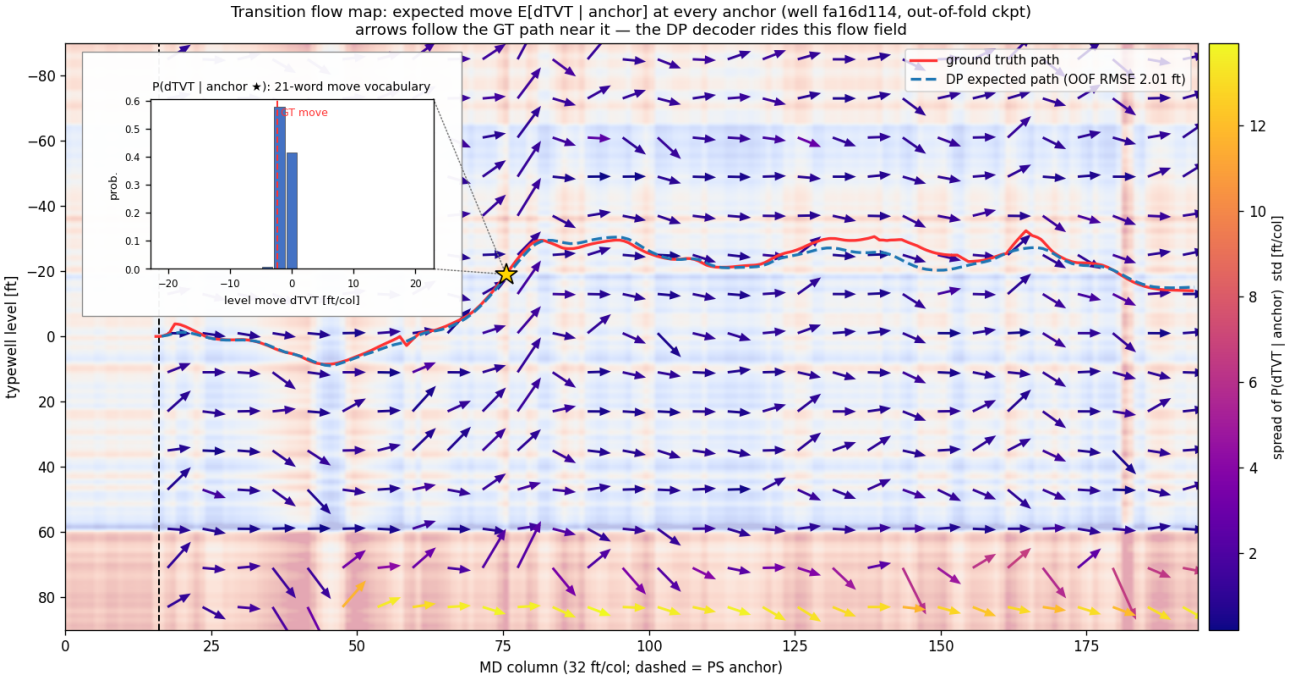
1. **Rollout**: sample stochastically from P(dTVT | anchor) to generate a bundle of random paths

2. **Path expectation (DP)**: marginalize the state grid with dynamic programming to compute the expectation of the path distribution exactly. Starting from a point mass at the PS level, the marginal p_k over levels is propagated column by column, and the prediction is its per-column mean:

```
p_{k+1}(s) = Σ_{s'} p_k(s') · P(dTVT = s − s' | anchor(s', k))     (5)
TVT_hat(k) = Σ_s p_k(s) · TVT(s)                                   (6)
```

No sampling is involved — (5)–(6) are computed exactly.

Initially I planned to generate N=1024 rollout paths and filter them with a reranker, but that turned out not to work, so the submissions use the "average path" computed by DP. For what it's worth, Viterbi (maximum-likelihood path) and other mode-tracking decoders were consistently worse than the expectation.



Ensembling and TTA

The final submission averages: 5 folds × 3 seeds = 15 checkpoints of DP paths × MD-phase TTA (8 views) × 2 grid resolutions (32 ft/column and 16 ft/column). The whole ensemble scores the 200 hidden test wells in about 40 minutes on the Kaggle T4.

MD-phase TTA: the MD resolution of the CNN input is 32 ft (one column), so a phase ambiguity below 32 ft — where exactly the columns are cut — remains. Relative to the GR correlation length (~18 ft), the per-column box averaging is coarse, and the input values themselves change depending on where you cut the columns (aliasing). To compensate, I ran inference at 8 sub-resolution phases, shifting the column-grid origin by {0, 8, 16, 24, 32, 40, 48, 56} ft, and averaged the predictions.

Robust validation and model selection

The competition metric, pooled RMSE, is proportional to the sum of squared errors, so it is easily dominated by "catastrophic wells". The metric moves far more when you fix a few badly failing wells than when you further improve wells that are already doing fine.

The question is whether an improvement on a few wells is systematic or just luck. For example, if the direction of improvement across the 5 dominant wells is random, it should be considered chance, not a real improvement.

At the same time, the data is small: 773 train wells, and only 148 wells in the private test. This makes statistically careful model selection essential.

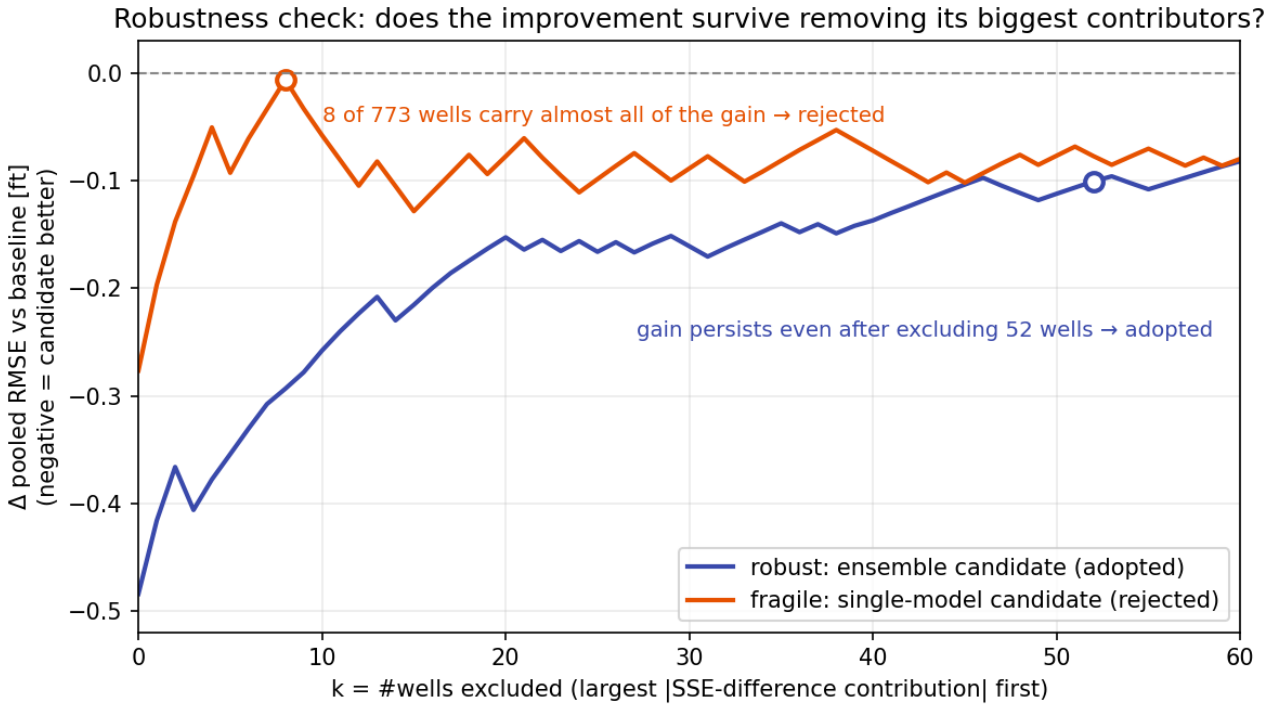
Robust acceptance test (leave-largest-contribution-out)

When comparing two models base and treat, compute the per-well difference of squared-error sums (SSE)

```
g_w = SSE_treat(w) − SSE_base(w)
```

and remove wells in descending order of |g_w| (i.e. the wells contributing most to the difference). Define k* as the number of removals at which the improvement first disappears. Unless k* was large enough — i.e. unless the improvement did not hinge on the luck of a few wells — I rejected the candidate.

The figure below shows a real example (removal curves of two candidates against the baseline). The **orange curve** appears to improve OOF by −0.28 at k = 0, but removing just 8 of 773 wells almost erases the gain — the improvement hinged on the luck of a few wells, so this candidate was rejected. The **blue curve** decays gently and its gain survives even after removing 52 wells (the size of the public LB), so this candidate was adopted.



Results

sub	configuration	OOF pooled	public	private
068	AnchorCNN (32 ft/col grid) × 15 ckpt	5.624	**5.780**	6.126
071	068 + MD-phase TTA8	5.417	5.820	6.078

sub	configuration	OOF pooled	public	private
075	16 ft/col grid × TTA8 alone (diagnostic)	5.348	5.970	5.882
076 (final, selected)	dz_layer-head models × TTA8 + 16 ft/col × TTA8	**5.140**	6.146	**5.802**
077 (final, hedge)	068-family × TTA8 + 16 ft/col × TTA8	5.242	5.910	5.905

- Final standing: **2nd place (private 5.802, sub076)**

Inference Code

What didn't work

I'll list only the rejections that became design forks.

Selecting from a path bundle (reranker, reinforcement learning): generating a rollout bundle (N=1024) does put very good paths inside the bundle — an oracle that picks the best path in hindsight beats the deployed score by several ft. Trying to capture that gap, I built a reranker (a model that picks good paths from the bundle) and tried reinforcement learning (GRPO), but both selectors overfit the evaluation data easily and never consistently beat the DP expectation. In the end I abandoned the "select from a bundle" design and committed to DP marginalization (expectation decoding).

Explicit formation modeling: modeling the formation surfaces themselves from neighboring wells captures the coarse topography of the field, but could not fix the catastrophically bad wells. My guess is that the teacher labels (expert interpretations) do not reference the formations directly — they are relative judgments based on GR matching. This result settled the design: no explicit formation model; commit entirely to the probability field over GR matching.

Final thoughts

What this model imitates is the interpretation work of geosteering experts — and the ambiguity in that work is intrinsic to the task. Rather than forcing the ambiguity to resolve prematurely, this approach keeps it alive as a probability distribution over the candidate moves (P(dTVT|TVT)). I believe this is its most practically valuable property: where the GR matching is ambiguous, the model can present alternative path hypotheses and flag low-confidence intervals for human review. And if independent external data (e.g., offset wells or seismic) becomes available, it can be used to reweigh the hypotheses and narrow down which of the ambiguous paths is the most plausible.

References

- [1] discussion: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/699853
- [2] Sergey Alyaev et al., "Direct Multi-Modal Inversion of Geophysical Logs Using Deep Learning" — https://arxiv.org/pdf/2201.01871 (poster: https://nfes.org/assets/workshop2022/ambrus_sequential_multi_mode_inversion_poster.pdf)

- [3] Comment by @pcjimmmy (pointing out that typewells can be consolidated into master series): https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/705210#3468651
- [4] Joseph Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection" — https://arxiv.org/abs/1506.02640
- [5] Xingyi Zhou et al., "Objects as Points" (CenterNet) — https://arxiv.org/abs/1904.07850
- [6] Comment by the host @igorakuvaev (pointing out that the lateral GR before the Prediction Start point has better resolution than the typewell GR, so it is better to use it for GR correlation): https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/698825#3456361

Supplemental Q&A

> **Tucker Arrants · 2026-08-06T21:38:59.783000**
>
> Thank you for sharing and congrats on 2nd place.
>
> Odd question - did Claude recommend the MD-Phase TTA to you with the 8 phases? Claude proposed the exact same formulation in my modeling trials with it and it was one of the few really good novel ideas it proposed when I ran out of my own.
>
> Just wondering if our Claudes converged on a similiar TTA idea here or if that was your own creation. It seems obvious in hindsight but I never would have thought of it myself. A lot of people on Kaggle claim agents will just "converge to the same solution" so I am just collecting data points on that idea.

> **Bilzard · 2026-08-06T22:09:23.260000**
>
> @tuckerarrants
>
> Claude suggested the MD shift, but it was me who suggested the shift width should be sub-column (within the 32 ft column width, restoring it afterwards). The 8 phases came out of a sweep — neither of us picked that number by intuition. Claude suggested tons of ideas, but I sometimes felt overwhelmed processing all the information it gave me, and most of the ideas didn't look promising. (To be fair, it sometimes did a really great job on things I would never have come up with — but the overall quality of the discussion felt quite random to me.)
>
> I think today's "autonomous" agents aren't autonomous enough — they need a human alongside them, otherwise they get stuck on really elementary problems. I believe human engineers still have to carry the torch ourselves; otherwise we just end up burning time.